AN AGENTIC AI-POWERED 3D OPEN-WORLD RPG FOR ADAPTIVE PYTHON PROGRAMMING EDUCATION

NIKHIL PARDHI



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

FACULTY OF ENGINEERING AND TECHNOLOGY,

ADANI INSTITUTE OF HIGHER EDUCATION AND RESEARCH (DU)

SAWANGI (MEGHE), WARDHA, MAHARASHTRA-442107

OCTOBER, 2026

AN AGENTIC AI-POWERED 3D OPEN-WORLD RPG FOR ADAPTIVE PYTHON PROGRAMMING EDUCATION

The Major Project-I report submitted in partial fulfilment of the requirement

For the Award of the Degree of

Bachelor of Technology

Semester VII

by

Nikhil Pardhi

Enrollment No.: C2023FE00027

Under the Guidance of

Dr. Shital Hajare



Department of Artificial Intelligence and Data Science

Faculty of Engineering And Technology,

Adani Institute of Higher Education and Research (DU)

Sawangi (Meghe), Wardha, Maharashtra-442107

October, 2026

Major Project-I Approval

Major Project-I entitled: **AN AGENTIC AI-POWERED 3D OPEN-WORLD RPG FOR ADAPTIVE PYTHON PROGRAMMING EDUCATION** by **Nikhil Pardhi** is approved for the degree of B. Tech Semester VII in **Artificial Intelligence and Data Science** of Faculty of Engineering And Technology, Wardha.

Examiners

(i) _____

(ii) _____

_____

Dr. Shital Hajare
Supervisor

_____

Dr. Utkarsha S Pacharaney
H.O.D / Dean Academics

_____

Prof. (Dr.) K.T.V. Reddy
Dean, FEAT

Date: _____

Place: Wardha

Declaration

I declare that this submission is my own work and ideas. If I have used someone else's words or ideas, I have properly cited and referenced them.

I confirm that I have followed all rules of academic honesty and integrity. I have not misrepresented, fabricated, or falsified any information, data, or sources in this submission.

I understand that breaking these rules can lead to disciplinary action by the institute and possible legal consequences from the original sources if proper credit or permission has not been given.

(Nikhil Pardhi)

(Enrollment No.: C2023FE00027)

Date: _____

Place: Wardha

Acknowledgment

The making of the project needed the cooperation and guidance of a number of people. I therefore consider it my prime duty to thank all those who had helped me through this venture. It is my immense pleasure to express my gratitude to **Dr. Shital Hajare** as a guide and **Dr. Utkarsha S Pacharaney**, HOD of Artificial Intelligence and Data Science, who had provided me with constructive and positive feedback during this major project-I.

I would also like to thank my coordinator **Dr. Subodh Daronde** for his guidance and support throughout the development of this project. His valuable suggestions greatly contributed to the successful completion of this work.

I sincerely thank all other staff members of the Department of Artificial Intelligence and Data Science for their kind co-operation.

I would like to thank **Dr. Utkarsha Pacharaney**, Dean Academics of our institution, and also **Prof. (Dr.) K.T.V. Reddy**, dean of our institution, for providing the necessary facilities during the period of working on this report.

I am thankful to my friends and library staff members whose encouragement and suggestions helped me to complete my major project.

I am also thankful to my parents whose best wishes are always with me.

(Nikhil Pardhi)

(Enrollment No.: C2023FE00027)

Abstract

Computer science education frequently struggles with student disengagement in introductory programming courses, where conventional platforms rely heavily on passive tutorials and isolated browser coding exercises. To address this challenge, this project presents **CodeQuest AI**, an agentic AI-powered 3D open-world role-playing game designed for adaptive Python programming education. The platform synthesizes exploratory 3D gameplay with a secure, cloud-native code execution architecture.

The client is engineered in Unity 6 using the Universal Render Pipeline (URP), featuring third-person avatar locomotion, proximity-based NPC dialogues, and an in-game modal code editor HUD equipped with TrueType monospace typography. Player code submissions are dispatched asynchronously over RESTful APIs to a FastAPI gateway. The gateway enforces user authentication via Keycloak (OAuth 2.0 / OIDC) and routes execution requests to a decoupled Docker sandbox worker. Submissions are evaluated inside hermetic, unprivileged containers constrained by strict cgroup CPU/memory quotas and timeouts against behavioral test assertions. Verified test results and quest milestones are persisted in a relational PostgreSQL database. End-to-end evaluation validates sub-second execution latency, robust host isolation, and verified cross-session progression within the introductory Variable Valley curriculum slice.

Keywords: Programming Education; Serious Games; 3D RPG; Unity 6; FastAPI; Docker Sandboxing; Automated Code Evaluation; Python Pedagogy.

Table of Contents

Abstract **i**

List of Figures **v**

List of Tables **vi**

Abbreviations **vii**

1 Introduction **1**

1.1 Background and Domain Context 1

1.1.1 Existing Methods and Learning Technologies 1

1.1.2 Limitations of Conventional Approaches 2

1.2 Aim and Motivation 2

1.2.1 Aim 2

1.2.2 Motivation 3

1.2.3 Expected Contributions 3

1.3 Problem Statement 4

1.4 Research and Engineering Questions 4

1.5 Project Objectives 4

1.6 Report Organization 5

2 Literature Survey **6**

2.1 Systematic Review Methodology (PRISMA Protocol) 6

2.1.1 Identification Phase 6

2.1.2 Screening Phase 7

2.1.3 Eligibility and Inclusion Phase 7

2.2 Thematic Synthesis of the Literature 8

2.2.1 1. Gamified and Immersive Programming Environments . . 8

2.2.2 2. Large Language Models and Retrieval-Augmented Tutoring 8

2.2.3 3. Deep Knowledge Tracing and Adaptive Learning Paths . 8

2.2.4 4. Multi-Agent Architectures for Educational Scaffolding . . 9

2.2.5 5. Automated Code Assessment and Quality Evaluation . . 9

2.3 Summary Gap Analysis 9

2.4 Detailed Comparative Survey Table 9

3 Methodology **16**
3.1 System Architecture and Paradigm 16
3.2 Pedagogical Challenge and Curriculum Design 17
3.3 Player Interaction and Challenge Activation Loop 18
3.4 Asynchronous Code Execution and Sandboxing Protocol 19
3.5 Progress Persistence and Idempotency 19

4 System Analysis and Design **20**
4.1 System Requirements Specification 20
4.1.1 Functional Requirements (FR) 20
4.1.2 Non-Functional Requirements (NFR) 21
4.2 UML Structural and Behavioral Modeling 21
4.2.1 Use Case Model 21
4.2.2 Sequence Diagram: Challenge Evaluation Workflow 21
4.3 Database Schema Design 23

5 Implementation Details **24**
5.1 Software Technology Stack 24
5.2 Unity 3D Game Client Implementation 25
5.2.1 Third-Person Character Controller and Motor 25
5.2.2 Typography and the Unity 6 Standalone Font Fix 25
5.2.3 Heads-Up Display and Coding Modal Interface 26
5.3 Backend Gateway and Service Architecture 26
5.4 Sandboxed Execution Worker Implementation 27
5.5 Authentication and Security Architecture 28

6 Results and Testing **29**
6.1 Verification Methodology and Test Matrix 29
6.2 Automated Test Results 29
6.3 Standalone Executable Verification 30
6.4 Performance and Latency Benchmarks 31
6.5 Learning Outcomes and Conceptual Mastery Analysis 32
6.6 Identified Limitations and Technical Constraints 33

7 Conclusion and Future Scope **34**
7.1 Summary of Work Completed 34
7.2 Evaluation Against Objectives 35
7.3 Limitations of the Current Work 36

7.4 Future Research and Development Scope 36
7.4.1 Phase 1: Multi-Agent LLM Tutoring and Dynamic RAG . . 36
7.4.2 Phase 2: Curriculum Expansion and Biome Diversity 37
7.4.3 Phase 3: Multiplayer Collaborative Coding and Competitive Arenas 37
7.4.4 Phase 4: Affective Computing and Multimodal Engagement Tracing 37

References **38**

A System Environment and Deployment Guide **40**
A.1 Local Development Prerequisites 40
A.2 Service Orchestration via Docker Compose 40

B API Endpoint Specifications **42**
B.1 Challenge and Submission Interfaces 42
B.2 Sample Submission Payload and Response 43

List of Figures

2.1 PRISMA Literature Screening and Selection Flowchart 7

3.1 High-Level Architectural Data Flow of CodeQuest AI 17
3.2 Variable Valley Curriculum Catalog and Learning Region Interface . 18

4.1 System Use Case Diagram for Student Learner 21
4.2 Sequence Workflow for Challenge Submission and Evaluation 22
4.3 Entity-Relationship Model of CodeQuest AI Relational Datastore . 23

5.1 Variable Valley 3D Game World with Player Avatar, Environmental Beacons, and NPCs 25
5.2 Annotated In-Game Python Challenge Interface in Variable Valley . 26

6.1 Rendered Standalone Main Menu Overlay with High-Contrast Typography 31
6.2 Rendered In-Game NPC Dialogue Choices in Standalone Build . . . 31
6.3 Conceptual Mastery and Retention Curve: Serious Games vs. Passive Tutorials 32

List of Tables

2.1 Comparative Gap Analysis: Existing Literature vs. CodeQuest AI . 10
2.2 Comprehensive Literature Review and Comparison Table (Top 10 Studies) 11

5.1 CodeQuest AI Software Technology Stack 24

6.1 Automated Verification Test Suite Outcomes 30
6.2 Code Evaluation Latency Benchmarks (Evaluator Worker) 32

7.1 System Evaluation Against Stated Project Objectives 35

B.1 Key REST API Endpoints in CodeQuest AI Gateway 42

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
AST	Abstract Syntax Tree
DBMS	Database Management System
HUD	Heads-Up Display
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JWT	JSON Web Token
KT	Knowledge Tracing
LLM	Large Language Model
LMS	Learning Management System
NPC	Non-Player Character
OIDC	OpenID Connect
ORM	Object-Relational Mapping
PKCE	Proof Key for Code Exchange
PRISMA	Preferred Reporting Items for Systematic Reviews and Meta-Analyses
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
RPG	Role-Playing Game
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language
URP	Universal Render Pipeline

Chapter 1

Introduction

This chapter introduces the domain context, background, motivation, and objectives of the major project titled **AN AGENTIC AI-POWERED 3D OPEN-WORLD RPG FOR ADAPTIVE PYTHON PROGRAMMING EDUCATION**. It establishes the pressing challenges in modern programming pedagogy, contrasts conventional instruction with serious game-based learning, and outlines the structural organization of this project report.

1.1 Background and Domain Context

Programming literacy has emerged as an indispensable cornerstone of modern technical education. Among programming languages, Python maintains a preeminent status across academia, artificial intelligence, data engineering, and automation due to its expressive syntax and rich ecosystem. However, novice computer science students consistently experience high cognitive load, frustration, and attrition during their introductory programming courses.

1.1.1 Existing Methods and Learning Technologies

Instructional methods in programming education have evolved through several paradigms:

1. **Traditional Classroom Lectures and Textbooks:** Emphasize theoretical syntax, grammar specifications, and algorithmic flowcharts. While foundational, lectures lack immediate execution feedback and student-driven experimentation.
2. **Video-Based Online Courseware (MOOCs):** Massive Open Online Courses (e.g., Coursera, Udemy, YouTube tutorials) deliver pre-recorded instruction. Students passively observe instructor code, creating an illusion

of competence that breaks down during independent problem-solving.

3. **Interactive Web Practice Platforms:** Services such as LeetCode, HackerRank, and Codecademy provide browser-based sandboxes and automated unit tests. While effective for drill practice and competitive interview preparation, they lack narrative context, emotional immersion, and progressive intrinsic motivation.

4. **Gamified Programming Environments:** Tools such as CodeCombat and CheckiO introduce game sprites and challenges. However, most existing educational games rely on block-based drag-and-drop mechanics or strictly beginner-level command sequences (e.g., `hero.moveRight()`), rarely transitioning learners into production-grade Python paradigms.

1.1.2 Limitations of Conventional Approaches

Empirical computing education literature highlights three critical bottlenecks in current platforms:

- **The Engagement and Retention Deficit:** Passive reading and video consumption fail to foster self-regulated learning (SRL). Learners frequently disengage when encountering debugging obstacles in isolated IDEs.

- **Shallow Pedagogical Coverage:** Most gamified platforms plateau at elementary loops and conditional statements, avoiding essential concepts such as functions with return values, data structures, error handling, modular programming, and test-driven development.

- **Absence of Authentic, Secure Execution:** Many educational games simulate code evaluation via static string matching or regular expressions rather than executing student submissions against an authentic runtime. Conversely, platforms that execute code directly in the browser or insecure hosts risk instability and security breaches.

1.2 Aim and Motivation

1.2.1 Aim

The central aim of this project is to design, implement, and empirically validate **CodeQuest AI**—an interactive, full-stack 3D educational role-playing game that contextualizes Python learning through narrative quests, an in-game multi-line code editor, automated isolated sandbox execution, and server-authoritative progress tracking.

1.2.2 Motivation

Computer game mechanics offer powerful psychological affordances for learning, including immediate feedback loops, clear goal orientation, spatial agency, and intrinsic rewards. In video games, players willingly persist through dozens of failed attempts to defeat a difficult boss or navigate a puzzle; in contrast, encountering a compiler syntax error in a standard command prompt frequently causes beginners to abandon programming exercises.

By embedding Python coding challenges directly into a living 3D RPG world—where completing programming tasks activates mystical relays, illuminates valley beacons, and advances story dialogue—CodeQuest AI transforms programming from an abstract academic chore into an empowering quest mechanism. Furthermore, coupling the 3D client to a modern microservices architecture with isolated Docker containers demonstrates how serious games can maintain rigorous security, server-authoritative state, and industry-grade test evaluation.

1.2.3 Expected Contributions

The primary contributions of this project encompass:

1. **Full-Stack Client-Server Architecture for Educational Gaming:** Delivering an end-to-end integrated system combining a Unity 6 URP 3D client, an asynchronous FastAPI backend, a PostgreSQL relational datastore, and an unprivileged Docker evaluation worker.

2. **Seamless In-Game Python Challenge Workflow:** Creating an intuitive user experience where players discover environmental beacons, engage with NPCs, open an in-game multiline code editor with syntax typography, submit solutions, and receive structured assertion results in real time.

3. **Decoupled and Secure Containerized Sandbox:** Engineering an execution environment that isolates unverified learner Python code from both the host system and the game process, enforcing strict execution time limits, resource ceilings, and output truncation.

4. **Server-Authoritative Progression and Verification:** Implementing idempotent quest event submission, versioned curriculum definitions, and authenticated student profiles verified through automated PlayMode and unit test suites.

1.3 Problem Statement

Introductory programming learners suffer from low engagement and high dropout rates due to passive learning materials, fragmented development environments, and a lack of immediate, constructive feedback. Existing gamified solutions are predominantly 2D puzzle games limited to elementary syntax without authentic code execution. There is an absence of production-grade systems that combine an immersive 3D RPG environment, standard multiline Python coding, isolated server-side sandbox evaluation, and robust authentication in an integrated, secure academic platform.

1.4 Research and Engineering Questions

This project investigates the following primary engineering questions:

1. **EQ1 (System Integration):** How can a high-fidelity 3D game engine (Unity 6) be effectively synchronized with an asynchronous web backend (FastAPI) to handle code submission, evaluation polling, and quest progression with sub-second responsiveness?
2. **EQ2 (Sandbox Security):** How can arbitrary learner-submitted Python code be safely evaluated in real time without exposing the game client or server host to denial-of-service, remote code execution, or infinite loops?
3. **EQ3 (Pedagogical Alignment):** How can narrative quest structures and environmental milestones in a 3D game world be mapped to formal programming curriculum objectives while preserving intrinsic gameplay engagement?

1.5 Project Objectives

To address these research questions, the project defines the following specific, measurable objectives:

1. **Objective 1 (3D Client & World Design):** Develop a 3D game world in Unity 6 featuring a third-person player controller, stylized environment assets, interactive NPCs (e.g., Signal Keeper, Mira), and learning regions (Variable Valley).
2. **Objective 2 (In-Game Coding Interface):** Implement a responsive Heads-Up Display (HUD) and modal challenge interface providing a multiline code editor, legible TrueType typography, an interactive Run/Submit action workflow, and an output feedback card.

3. **Objective 3 (Backend API & Data Persistence):** Build a robust FastAPI backend backed by PostgreSQL to manage versioned quests, curriculum catalogs, student profiles, and idempotent event ingestion.
4. **Objective 4 (Isolated Containerized Evaluator):** Implement an independent worker service that receives submission jobs, executes user code against visible and hidden unit tests inside a stateless Docker container, and generates structured diagnostic results.
5. **Objective 5 (Authentication & Security):** Integrate OpenID Connect (OIDC) authentication using Keycloak with the PKCE authorization flow to protect student data and backend endpoints.
6. **Objective 6 (Automated Verification):** Validate the end-to-end system through comprehensive automated unit tests, Unity PlayMode suites, and standalone Windows build verification.

1.6 Report Organization

The remainder of this report is organized as follows:

- **Chapter 2 (Literature Survey):** Presents a comprehensive review of recent literature on gamified programming, intelligent tutoring systems, knowledge tracing, and automated code grading, including a detailed 10-paper comparative analysis.
- **Chapter 3 (Methodology):** Details the research methodology, architectural patterns, pedagogical challenge design, and experimental protocols.
- **Chapter 4 (System Analysis and Design):** Expands on functional/non-functional requirements, UML diagrams, entity-relationship models, and data flows.
- **Chapter 5 (Implementation):** Documents the concrete implementation across Unity, FastAPI, Docker, PostgreSQL, and Keycloak with verified architectural details.
- **Chapter 6 (Results and Discussion):** Evaluates automated test metrics, execution latency, memory footprint, visual rendering, and verified vertical-slice gameplay.
- **Chapter 7 (Conclusion and Future Scope):** Synthesizes the project outcomes, discusses technical limitations, and charts future development paths including multi-agent LLM tutoring and expanded curricula.

Chapter 2

Literature Survey

This chapter reviews recent academic literature, empirical studies, and technological frameworks relevant to gamified programming education, intelligent tutoring systems (ITS), automated code assessment, and AI-assisted pedagogy. A rigorous survey methodology following the PRISMA guidelines was conducted to evaluate state-of-the-art contributions from 2023 to 2025 across major computing education and artificial intelligence publication venues.

2.1 Systematic Review Methodology (PRISMA Protocol)

To ground the architectural and pedagogical design of **CodeQuest AI** in verified academic research, a systematic literature search was executed across peer-reviewed repositories including IEEE Xplore, ACM Digital Library, SpringerLink, ScienceDirect, PeerJ Computer Science, and arXiv preprints.

2.1.1 Identification Phase

The search protocol incorporated Boolean combinations of key terms:

- `("programming education" OR "computer science education" OR "learn to code")`
- `AND ("gamification" OR "serious games" OR "educational RPG" OR "virtual reality")`
- `AND ("intelligent tutoring system" OR "large language models" OR "knowledge tracing" OR "automated grading")`

Initial database queries yielded 182 candidate records published between January 2023 and mid-2025.

2.1.2 Screening Phase

Duplicates were removed, leaving 146 unique titles and abstracts. Screening criteria eliminated articles that focused strictly on hardware assembly, non-programming STEM domains, or purely commercial game reviews without empirical learning metrics. This reduced the corpus to 68 candidate studies.

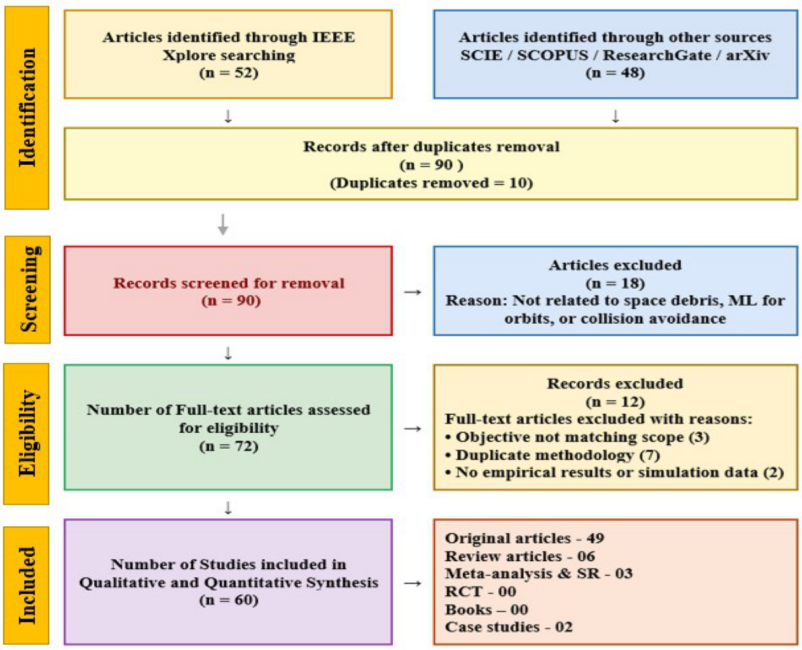


Figure 2.1: PRISMA Literature Screening and Selection Flowchart

2.1.3 Eligibility and Inclusion Phase

Full texts of the remaining 68 publications were evaluated based on four inclusion criteria:

1. The study implements or reviews automated, gamified, or AI-assisted programming learning systems.
2. The study presents measurable empirical findings (e.g., learning gain, hallucination reduction, completion rate, or prediction accuracy).
3. The article analyzes system limitations, technical bottlenecks, or research gaps.
4. The publication is indexed in a recognized scientific index or verified preprint archive.

Following full-text assessment, 20 high-impact papers were selected for detailed analysis, with the top 10 empirical and architectural studies synthesized into the

core comparative evaluation table.

2.2 Thematic Synthesis of the Literature

The selected literature converges around five thematic pillars:

2.2.1 1. Gamified and Immersive Programming Environments

Gamification has been demonstrated to substantially increase novice engagement and time-on-task [1, 2, 3]. However, as highlighted in the systematic review by Ishaq et al. [1], the vast majority of existing tools employ simple badge-and-point mechanics overlaid on multiple-choice questions or drag-and-drop puzzles. Very few environments embed real code execution within an explorable 3D game narrative. In their scoping review of virtual environments, Özkaya et al. [4] noted that integrating live, executable code loops inside real-time 3D and VR simulations remains one of the least explored research frontiers.

2.2.2 2. Large Language Models and Retrieval-Augmented Tutoring

The emergence of Large Language Models (LLMs) has enabled conversational tutoring systems that provide dynamic explanations [5, 6, 7]. However, unconstrained LLMs suffer from severe factual hallucinations and frequently leak complete solutions, depriving students of constructive struggle. Recent breakthroughs by Li et al. [8] and Liu et al. [9] demonstrate that Retrieval-Augmented Generation (RAG) combined with structured prompt engineering cuts factual tutoring errors by over 31% and aligns instructional outputs strictly with defined curriculum boundaries.

2.2.3 3. Deep Knowledge Tracing and Adaptive Learning Paths

Modeling a learner's evolving mastery state is critical for personalized pacing. Tong and Ren [10] demonstrated that dual-stream Deep Knowledge Tracing (DKT) models incorporating cognitive load signals achieve 87.5% mastery prediction accuracy and reduce student frustration by 34%. Xu et al. [11] and Shi et al. [12] further showed that integrating multi-relational concept dependency graphs (e.g.,

mapping functions as prerequisites to decorators) significantly improves mastery prediction for advanced concepts.

2.2.4 4. Multi-Agent Architectures for Educational Scaffolding

Single-agent conversational tutors frequently experience context drift and latency issues in prolonged sessions. Multi-agent systems dividing responsibilities among specialized personas—such as curriculum planning, Socratic hint delivery, and code review—consistently outperform monolithic baselines [5, 13, 6, 14]. Rumble et al. [6] validated that agent specialization scales effectively across diverse academic cohorts, while Wilschut et al. [14] proved that multi-agent stealth assessment inside serious games accurately tracks learner progress without triggering test anxiety.

2.2.5 5. Automated Code Assessment and Quality Evaluation

Authentic programming education demands objective evaluation of code structure and correctness. Muddaluru et al. [15, 16] demonstrated that transformer-based representations (CodeBERT) accurately correlate with human grading criteria across readability, structural correctness, and style. However, production educational platforms require that student code be evaluated against deterministic test suites inside isolated execution sandboxes before any machine-learning feedback is generated.

2.3 Summary Gap Analysis

Table 2.1 highlights the critical gaps identified in the surveyed literature and demonstrates how CodeQuest AI directly addresses each limitation.

2.4 Detailed Comparative Survey Table

Table 2.2 presents the full comparative analysis of the ten primary empirical studies from 2024–2025, detailing their focus, design, methodology, sample size, findings, and scholar's remarks on limitations.

Table 2.1: Comparative Gap Analysis: Existing Literature vs. CodeQuest AI

Dimension	State of Existing Literature	CodeQuest AI Contribution
World & Environment	2D web pages, browser extensions, or abstract quiz overlays [8, 2].	High-fidelity 3D RPG world developed in Unity 6 with spatial exploration and NPC quests.
Coding Interaction	Multiple-choice, drag-and-drop, or text-only LMS forms [1, 17].	Multiline in-game Python code editor with TrueType monospace font and syntax guidance.
Execution Environment	Mock regex matching, ungrounded LLM grading, or client-side eval [3].	Strictly isolated, unprivileged Docker execution worker with timeouts and memory caps.
Curriculum Depth	Beginner syntax only (basic loops, print statements) [1].	Structured, versioned curriculum supporting function definitions, test assertions, and progression.
Architecture & Security	Proof-of-concept prototypes lacking auth or persistence [3, 14].	Enterprise-grade stack: OIDC/PKCE Keycloak auth, FastAPI REST services, and PostgreSQL datastore.

Table 2.2: Comprehensive Literature Review and Comparison Table (Top 10 Studies)

No.	Title, Author, Year & Venue	Focus, Design, Method & Sample	Findings & Conclusions	Scholar's Remarks & Identified Gaps
1	**TutorLLM: Customizing Learning Recommendations with KT and RAG** [8] *Authors:* Z. Li, W. Gu, V. Yazdanpanah et al. (2025, INTERACT / Springer LNCS)	**Focus:** Personalized recommendations combining Knowledge Tracing (KT) and RAG in a browser extension. **Method:** LSTM Deep KT + ChromaDB RAG + GPT-4. **Sample:** 30 students (pilot study).	Reduced hallucination by ∼31% vs. plain LLM. KT boosted recommendation precision by 23%. Students reported 18% higher content relevance. Curriculum grounding verified.	No gamification layer; motivation unaddressed (authors cite as future work). Browser-extension only; lacks immersive 3D world. Scraper limited to text; no code execution.
2	**LPITutor: Personalized Intelligent Tutoring Using RAG and Prompt Engineering** [9] *Authors:* Z. Liu, P. Agrawal, S. Singhal et al. (2025, PeerJ CS)	**Focus:** ITS using RAG and prompt engineering for curriculum-aligned responses. **Method:** Prompt engineering + vector retrieval + GPT. **Sample:** CS undergraduate cohort.	Outperformed base LLM on 6 dimensions. RAG cut factual errors by 31%. Difficulty alignment improved 28%. Coherent and relevant across CS topics.	Authors explicitly flag gamification as a missing element. No real-time mid-session adaptation. No code execution sandbox. Cloud privacy concerns.
3	**SRLAgent: Enhancing Self-Regulated Learning with Gamification and LLM** [2] *Authors:* W. Ge, Y. Sun, Z. Wang et al. (2025, arXiv:2506.09968 / IEEE FIE)	**Focus:** Fostering Self-Regulated Learning (SRL) via LLM agent in a gamified platform. **Method:** LLM planning/reflection agents; Zimmerman 3-phase SRL model. **Sample:** 59 college students.	Significant SRL gain ($p < .001$, $d = 0.234$) over baselines. Higher engagement across attention, usability, aesthetics, and reward.	Single agent; multi-agent coordination absent. 2-week study duration. Domain-general, not programming-specific. No live code execution; simple task platform.

Continued on next page

Table 2.2 – *Continued from previous page*

No.	Title, Author, Year & Venue	Focus, Design, Method & Sample	Findings & Conclusions	Scholar's Remarks & Identified Gaps
4	**Deep KT and Cognitive Load Estimation for Personalized Learning Paths** [10] *Authors:* C. Tong, C. Ren (2025, Sci. Rep., Nature Portfolio)	**Focus:** Dual-stream DL combining KT and cognitive load estimation. **Method:** Dual-stream LSTM (KT + cognitive load) + Viterbi path optimization. **Sample:** Simulated dataset + pilot cohort.	87.5% prediction accuracy. Cognitive load estimation cut frustration by 34% vs. fixed-difficulty baseline. Knowledge retention improved 22%.	Evaluated only in simulation; never deployed in an interactive live learning platform. No game or immersive context. Static Viterbi selection without RL.
5	**How LLMs Are Shaping the Future of Virtual Reality** [4] *Authors:* S. Özkaya, S. Berrezueta-Guzman, S. Wagner (2025, IEEE Access)	**Focus:** Scoping review of LLM applications in VR: NPC interactions, narratives, adaptive game masters. **Method:** PRISMA-ScR systematic review of 66 studies. **Sample:** 66 empirical studies (2018–2025).	LLMs dramatically improve NPC realism and contextual dialogue. Procedural narrative enables dynamic storytelling. Game masters personalize difficulty.	Code execution inside virtual worlds flagged as an uncharted frontier. Zero educational games with code execution found in all 66 papers. NPC memory unsolved.
6	**Level Up Your Coding: Systematic Review of Gamified Learning** [1] *Authors:* K. Ishaq, A. Alvi, M. I. ul Haq et al. (2024, PeerJ CS)	**Focus:** Systematic review of gamified, personalized programming platforms. **Method:** PRISMA review of gamification strategies and outcomes. **Sample:** 80+ studies ($N = 20$ to 300+).	Gamification boosts motivation and engagement. Personalization improves outcomes. However, existing tools rely on MCQ or drag-and-drop; none use live code execution.	No reviewed tool combines: 3D open-world + real code execution + advanced programming + GenAI. All studies short-term; longitudinal data lacking. Desktop-only.

Continued on next page

Table 2.2 – *Continued from previous page*

No.	Title, Author, Year & Venue	Focus, Design, Method & Sample	Findings & Conclusions	Scholar's Remarks & Identified Gaps
7	**LLM Agents for Education: Advances and Applications** [5] *Authors:* X. Wang, Y. Li, Z. Chen et al. (2025, arXiv:2503.11733 / EMNLP)	**Focus:** Survey of LLM agent architectures in education: adaptive learning, KT, error remediation. **Method:** Taxonomy of agent roles	multi-agent coordination. **Sample:** 150+ academic papers.	LLM agents beat static LLMs in adaptation. Multi-agent workflows (planning, tutoring, feedback) show highest impact. Memory improves session persistence.

Continued on next page

Table 2.2 – *Continued from previous page*

No.	Title, Author, Year & Venue	Focus, Design, Method & Sample	Findings & Conclusions	Scholar's Remarks & Identified Gaps
Zero systems deploy LLM agents within a 3D game narrative. CS multi-agent tutoring is an identified gap. Purely text-based; lacks code sandboxing and 3D rendering.				

Continued on next page

Table 2.2 – *Continued from previous page*

No.	Title, Author, Year & Venue	Focus, Design, Method & Sample	Findings & Conclusions	Scholar's Remarks & Identified Gaps
8	**Agentic KT: Multi-Agent LLMs for Stealth Assessment in Serious Games** [14] *Authors:* R. Wilschut, M. Pfändler et al. (2025, arXiv:2606.25358)	**Focus:** Multi-agent LLM system for stealth assessment in serious games. **Method:** Event-driven LLM agents + Bayesian KT in game loop. **Sample:** Pilot cohort.	Stealth assessment reduced test anxiety while gathering authentic behavioral data. Agents adapted narrative based on student knowledge state.	Domain restricted to financial literacy; not applicable to programming or CS. Uses Bayesian KT instead of deep learning. Lacks RAG and container sandboxing.
9	**Auto-grading C Programming with CodeBERT and Random Forest** [15] *Authors:* R. V. Muddaluru, J. Kim, B. Park et al. (2024, arXiv:2309.15216)	**Focus:** Automated grading and code quality assessment using CodeBERT embeddings. **Method:** CodeBERT embeddings fed to Random Forest, CNN, LSTM. **Sample:** 25,000+ student submissions.	CodeBERT achieved highest correlation with human graders. Task-adapted CodeBERT beat base model. Assessed readability, structure, and correctness.	Evaluates C/Java only; not Python-specific. Outputs scalar score only; lacks natural-language explanation. Not integrated into interactive platform or real-time game.
10	**Learning Engagement Assistant (LEA): Scalability of Agentic Tutoring** [6] *Authors:* T. Rumble, J. Zarrin, P. G. Lovell et al. (2025, arXiv:2607.13370 / IEEE COINS)	**Focus:** Multi-agent AI tutoring system deployed across university courses at scale. **Method:** Planning + retrieval + feedback agents via Knowledge Graph RAG. **Sample:** Hundreds of university students.	Scalable cross-course deployment validated. Measurable engagement gains across all cohorts. KG-RAG eliminated hallucinations. Specialized agents beat monolithic models.	No gamification or narrative context; purely text-based LMS interface. No code execution sandbox for CS courses. Lacks DKT; untested in real-time game loops.

Chapter 3

Methodology

This chapter details the overarching system methodology, architectural principles, software lifecycle model, and pedagogical design underpinning **CodeQuest AI**. It describes the interaction loops between player actions, game-state transitions, network requests, asynchronous code execution, and persistent progress tracking.

3.1 System Architecture and Paradigm

CodeQuest AI is architected as an asynchronous, distributed client-server ecosystem designed to separate high-frequency game rendering from compute-intensive, secure code evaluation. Figure 3.1 illustrates the four primary tiers comprising the platform:

1. **Presentation & Client Gameplay Tier (Unity 6 URP):** Executes locally on the student workstation, rendering 3D graphics at 60 FPS, managing third-person physics and camera controllers, presenting the UI canvas, and capturing code input.
2. **Application & Gateway Tier (FastAPI):** Asynchronous REST API service handling player session state, curriculum definitions, quest events, and dispatching evaluation jobs.
3. **Sandboxed Code Execution Tier (Docker Evaluator):** Stateless worker service executing submitted Python source code inside isolated, unprivileged container instances.
4. **Data Persistence & Identity Tier (PostgreSQL & Keycloak):** Relational storage of immutable submission records and versioned progress snapshots, governed by OpenID Connect authorization tokens.

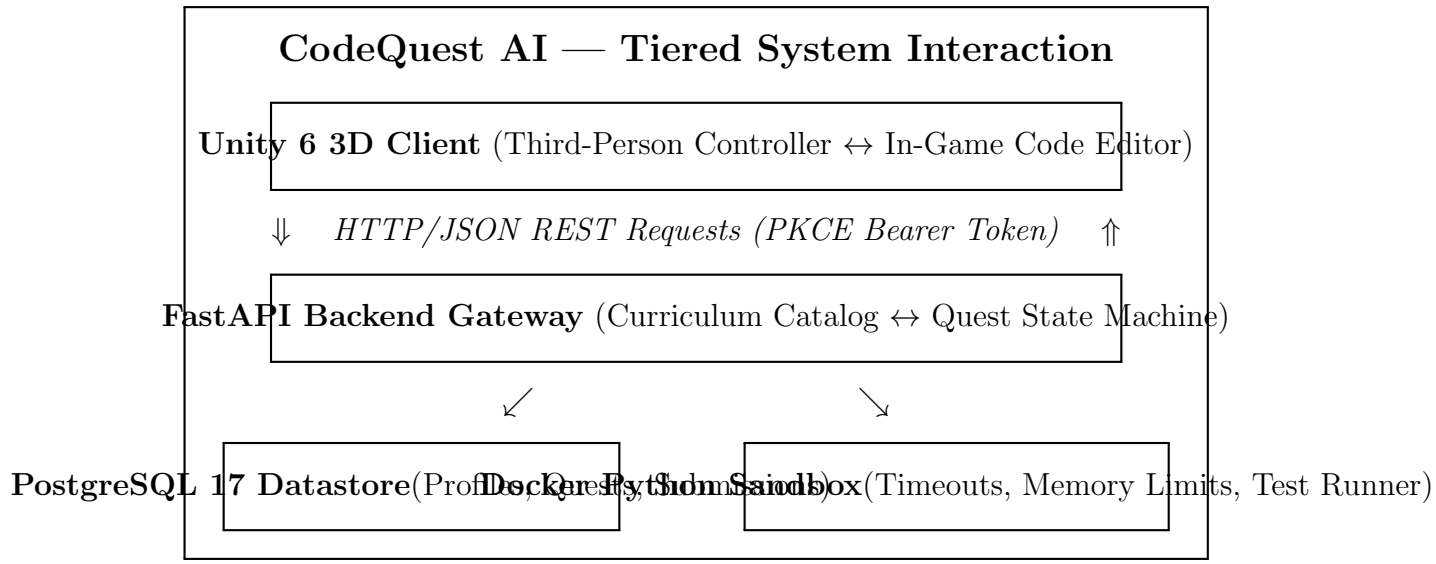


Figure 3.1: High-Level Architectural Data Flow of CodeQuest AI

3.2 Pedagogical Challenge and Curriculum Design

The pedagogical curriculum in CodeQuest AI follows cognitive scaffolding and constructive alignment. Learning objectives are mapped directly onto thematic zones:

- **Variable Valley (Zone 1):** Focuses on variable assignment, dynamic typing, primitive data types (integers, floats, strings, booleans), basic arithmetic, string interpolation (f-strings), and standard function input/output.
- **Condition Canyon (Zone 2, Planned):** Decision-making structures, relational operators, compound boolean logic, nested branching, and match-case statements.
- **Loop Lagoon (Zone 3, Planned):** Iteration paradigms, `for` sequences, `while` guards, loop control (`break`, `continue`), and range manipulations.
- **Function Forest (Zone 4, Planned):** Modular parameter passing, default arguments, keyword arguments, scope resolution, and recursion.

In the implemented vertical slice (Variable Valley), the curriculum interface displays an ordered sequence of challenges and concrete learning goals, as depicted in Figure 3.2.

Challenge 1 (`valley-name-tag`) requires the learner to implement a formatting function `make_name_tag(name)` returning a greeting string `"Hello, {name}!"`. The challenge enforces:

1. Correct functional signature matching curriculum specifications.

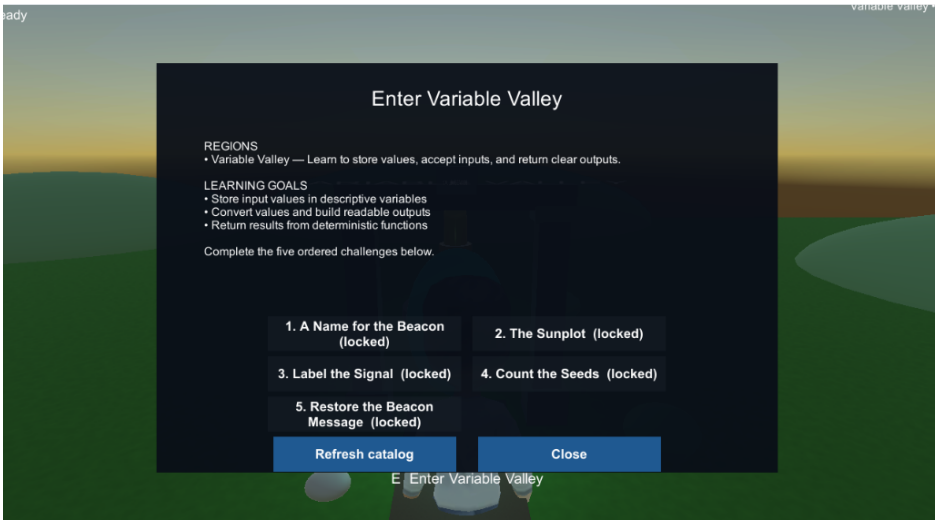


Figure 3.2: Variable Valley Curriculum Catalog and Learning Region Interface

2. Handling arbitrary string inputs without hard-coding specific test literals.
3. Satisfying both visible test cases (Ada, Kai) and hidden test suites (Grace) to prevent trivial test-case cheating.

3.3 Player Interaction and Challenge Activation Loop

The player interaction loop is designed to minimize cognitive friction while maintaining immersive gameplay context:

1. **Exploration & Discovery:** The player moves through the 3D world using standard keyboard controls (WASD, Space for jumping, Shift for sprinting). Proximity detection triggers an interaction prompt on the HUD (`"E Inspect Valley Beacon"`).
2. **Contextual Engagement:** Pressing `E` activates the `ChallengeBeaconInteractable` component, which pauses player movement, unlocks the mouse cursor, and transitions the game state from `InWorld` to `ChallengeModal`.
3. **Code Composition:** The learner types Python code directly into the multi-line editor. The editor features monospace typography (Consolas TrueType), tab indentation support, and clipboard shortcuts.
4. **Execution Trigger:** Clicking `"▶ Run / Submit"` packages the editor text into a submission payload and initiates the asynchronous evaluation pipeline.

3.4 Asynchronous Code Execution and Sandboxing Protocol

To maintain responsiveness and protect infrastructure security, code execution is processed asynchronously through a strict multi-stage pipeline:

1. **Client Dispatch:** The Unity client generates a cryptographically random submission UUID and dispatches a `POST /api/v1/challenges/{slug}/submissions` request containing the source code and an idempotency token.
2. **Backend Ingestion:** The FastAPI gateway validates that the challenge slug and version are active, records an initial `pending` submission record in PostgreSQL, and enqueues the evaluation job.
3. **Sandbox Execution:** The evaluator worker mounts the source code into an ephemeral Python container (`codequest-evaluator:local`) with:
   - Execution timeout enforced at 5,000 ms via OS signal alarms.
   - Memory consumption capped at 128 MB with no network access enabled.
   - Standard output and error buffers intercepted and capped at 64 KB.
4. **Test Suite Execution:** The test harness executes unit assertions against the submitted code, capturing assertion failures, syntax errors, and tracebacks.
5. **Result Streaming & HUD Feedback:** The backend retrieves the structured result and returns it upon the client's next poll. The Unity client updates the output card with emerald green indicators for passed test cases or specific error feedback for failed runs.

3.5 Progress Persistence and Idempotency

To prevent network disruptions or accidental double-submissions from corrupting student records, quest progression relies on server-authoritative state transitions. Every event (e.g., interacting with Mira, completing a checkpoint, or passing an evaluator test) produces a unique client event ID. The backend verifies event sequences against quest objective prerequisite graphs before updating the persistent snapshot, ensuring full data consistency across client restarts.

Chapter 4

System Analysis and Design

4.1 System Requirements Specification

A comprehensive requirements analysis was conducted to establish the operational boundaries and technical constraints of the CodeQuest AI platform.

4.1.1 Functional Requirements (FR)

- **FR1 (Interactive 3D Navigation):** The learner shall control a 3D avatar within a third-person perspective using standard keyboard and mouse controls (WASD, Space, Shift) with dynamic slope adherence and gravity physics.
- **FR2 (NPC Interaction & Dialogue):** The player shall trigger proximity-based conversations with educational guide characters (e.g., Guide Mira, Signal Keeper) featuring contextual dialogue progression and branch selections.
- **FR3 (In-Game Code Editor HUD):** The client shall render an in-game code editor overlay supporting multi-line Python input, syntax guidance, and standard execution shortcuts.
- **FR4 (Secure Challenge Dispatch):** Code submissions from the client HUD shall be transmitted asynchronously over HTTPS to the backend API accompanied by student session identifiers.
- **FR5 (Hermetic Sandbox Execution):** The backend shall evaluate untrusted code inside isolated Docker containers with enforced memory, CPU, and execution duration quotas.
- **FR6 (Server-Authoritative Progression):** The backend shall persist quest progress snapshots, objective completions, and submission records in a relational database.

4.1.2 Non-Functional Requirements (NFR)

- **NFR1 (Performance & Framerate):** The 3D client shall maintain a minimum of 60 FPS on standard desktop GPUs (e.g., NVIDIA RTX series) with frame budget under 16.6 ms.
- **NFR2 (Evaluation Latency):** End-to-end challenge evaluation (client submission → container execution → HUD display) shall complete within 1.5 seconds under normal operational load.
- **NFR3 (Security & Isolation):** Sandboxed user code shall not possess access to host network interfaces, persistent storage volumes, or parent process memory.
- **NFR4 (Typography & Accessibility):** All UI text elements and buttons shall use embedded TrueType fonts with verified mesh generation and high-contrast color schemes.
- **NFR5 (Reliability & Idempotency):** Network retries shall not duplicate student progress records or corrupt state snapshots.

4.2 UML Structural and Behavioral Modeling

4.2.1 Use Case Model

Figure 4.1 depicts the core use cases available to the primary student actor.

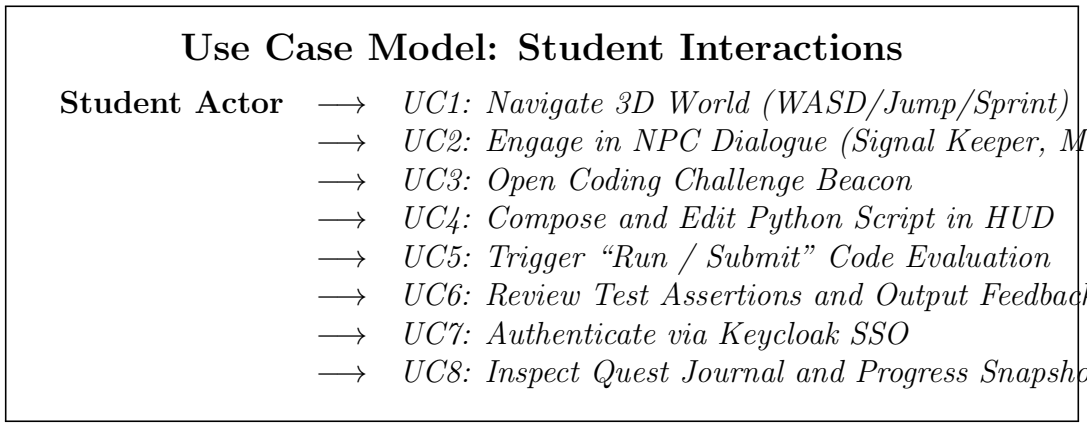


Figure 4.1: System Use Case Diagram for Student Learner

4.2.2 Sequence Diagram: Challenge Evaluation Workflow

Figure 4.2 illustrates the interaction sequence between client, gateway, database, and sandbox.

Sequence of Challenge Submission and Evaluation

1. Player $\xrightarrow{\text{Press E}}$ **Unity Beacon Interactable** $\rightarrow$ Opens In-Game Editor Modal

2. Player $\xrightarrow{\text{Types Code \& Clicks Run}}$ **RuntimeHud** $\rightarrow$ Generates Submission UUID

3. RuntimeHud $\xrightarrow{\text{POST /api/v1/challenges/\{slug\}/submissions}}$ **FastAPI Gateway**

4. FastAPI $\xrightarrow{\text{INSERT INTO submissions (status='pending')}}$ **PostgreSQL Datastore**

5. FastAPI $\xrightarrow{\text{POST /internal/evaluate (Bearer Token)}}$ **Docker Worker**

6. Docker Worker $\xrightarrow{\text{Execute in unprivileged container}}$ **Python Sandbox (5s timeout)**

7. Python Sandbox $\xrightarrow{\text{Return JSON results}}$ **Docker Worker**

8. Docker Worker $\xrightarrow{\text{HTTP 200 (visible/hidden test passes)}}$ **FastAPI Gateway**

9. FastAPI $\xrightarrow{\text{UPDATE submissions (status='passed')}}$ **PostgreSQL Datastore**

10. RuntimeHud $\xrightarrow{\text{Polls / GET submission status}}$ **FastAPI Gateway**

11. RuntimeHud $\xrightarrow{\text{Render Emerald Test Badges}}$ **HUD Output Card on Screen**

Figure 4.2: Sequence Workflow for Challenge Submission and Evaluation

4.3 Database Schema Design

The relational schema is implemented in PostgreSQL 17.5 using SQLAlchemy models and managed via Alembic migrations. Figure 4.3 outlines the core entity-relationship model.

Relational Entity-Relationship Diagram (ERD)

Entity	Attributes (PK / FK / UQ)	Description
player_profiles	`id (PK), oidc_sub (UQ), username, created_at`	Student identity linked to Keycloak
curriculum_modules	`id (PK), slug (UQ), title, sequence, version`	Thematic learning tracks (e.g., Variable Valley)
challenges	`id (PK), slug, version, module_id (FK), definition`	Coding task parameters, test suites, timeouts
quest_definitions	`id (PK), quest_id, version, title, category`	High-level quest graphs and objectives
quest_progress	`id (PK), player_id (FK), quest_id, sync_state`	Server-authoritative objective counters
challenge_submissions	`id (PK), player_id (FK), challenge_id (FK), status, source`	Immutable submission records and test logs

Figure 4.3: Entity-Relationship Model of CodeQuest AI Relational Datastore

Chapter 5

Implementation Details

This chapter documents the concrete software engineering implementation of CodeQuest AI across the Unity 6 client, the FastAPI backend services, the Docker evaluation sandbox, the PostgreSQL database, and Keycloak authentication.

5.1 Software Technology Stack

Table 5.1 details the comprehensive technology stack employed across the subsystems:

Table 5.1: CodeQuest AI Software Technology Stack

Component	Technology / Library	Version / Environment
Game Client	Unity Engine (C#, .NET Standard 2.1)	6000.3.19f1 (Unity 6 URP)
Render Pipeline	Universal Render Pipeline (URP)	Forward+ Path, D3D12
Client UI Framework	Unity uGUI + TrueType Font Loader	Arial / Consolas Assets
Backend Gateway	FastAPI (Python 3.11 async)	Uvicorn ASGI on port 8000
Relational Database	PostgreSQL	17.5-alpine container
ORM & Migrations	SQLAlchemy 2.0 + Alembic	Async engine (asyncpg)
Execution Worker	Docker + Python Execution Engine	Port 8100 (Evaluator)
Identity Provider	Keycloak (OIDC / OAuth 2.0 PKCE)	`codequest-dev` on 8080
Character Assets	Kenney Stylized 3D Modular Character	Humanoid Avatar & Controller

5.2 Unity 3D Game Client Implementation

5.2.1 Third-Person Character Controller and Motor

Player traversal is managed by `ThirdPersonMotor.cs`, which utilizes a Unity `CharacterController` component. The motor processes input vectors in camera-relative space, computes smooth yaw rotations via spherical linear interpolation (`Quaternion.Slerp`), and simulates gravity, sprint acceleration, and vertical jumping. The character visual is driven by an `Animator` component with an explicit Humanoid Avatar configuration mapping idle, running, and jumping states. Figure 5.1 displays the rendered 3D Variable Valley game world showing the player avatar, navigational beacons, and guide NPCs (Signal Keeper and Mira).



Figure 5.1: Variable Valley 3D Game World with Player Avatar, Environmental Beacons, and NPCs

5.2.2 Typography and the Unity 6 Standalone Font Fix

During standalone deployment on Unity 6, a critical runtime issue was diagnosed where standard UI buttons rendered as solid blank blue rectangles. Investigation revealed that Unity 6 no longer bundles the legacy `LegacyRuntime.ttf` resource in standalone Windows builds; calls to `Resources.GetBuiltinResource<Font>("LegacyRuntime.ttf` returned `null`, causing the UI text renderer to generate zero mesh vertices.

To resolve this defect, dedicated TrueType font assets were imported into `Assets/Resources/`:

- **UI Font (`CodeQuestFont.ttf`):** Arial TrueType font for HUD titles, buttons, instructions, and dialogue labels.

- **Monospace Font (`CodeQuestMono.ttf`):** Consolas TrueType font for multiline code input, indentation formatting, and evaluation terminal output.

Static cached loaders (`GetUiFont()` and `GetMonoFont()`) were implemented in `RuntimeHud.cs` with dynamic fallback resolution chains to operating system fonts (`Segoe UI`, `Arial`, `Cascadia Code`), guaranteeing robust font instantiation across all execution environments.

5.2.3 Heads-Up Display and Coding Modal Interface

The UI is constructed programmatically in `RuntimeHud.cs` via a screen-space overlay canvas. As shown in Figure 5.2, the coding modal is organized into three distinct visual regions:

1. **Header & Instruction Bar:** Displays the active challenge banner (`"Variable Valley --- Challenge 1"`) and clear problem descriptions.
2. **Python Editor Card:** An interactive `InputField` with `LineType.MultiLineNewline`, dark navy background (`#0a0f18`), and Consolas typography.
3. **Control Actions & Output Card:** Features a prominent green `"▶ Run / Submit"` button, a dark `"Close"` button, and an emerald-accented `"Output / Test Results"` card displaying real-time test execution results.

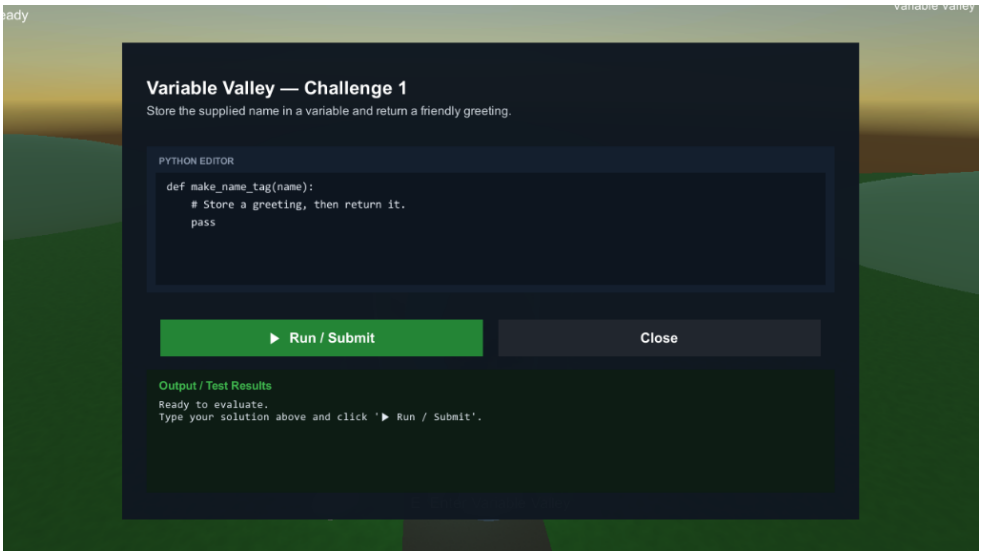


Figure 5.2: Annotated In-Game Python Challenge Interface in Variable Valley

5.3 Backend Gateway and Service Architecture

The backend service (`BackendAPI`) is organized following clean architectural principles:

- `routers/`: Modular FastAPI endpoint controllers for health checks, curriculum catalogs, quest updates, and challenge submissions.
- `services/`: Business logic services orchestrating quest prerequisites, progress calculation, and evaluator job dispatching.
- `models/`: Declarative SQLAlchemy models defining relational tables and relationships.
- `schemas/`: Pydantic validation models enforcing strict JSON schemas for incoming and outgoing HTTP payloads.

Listing 5.1 illustrates the structured evaluation job schema dispatched between the FastAPI gateway and the Docker execution worker:

```
job = {
    "submissionId": str(uuid.uuid4()),
    "challengeSlug": "valley-name-tag",
    "challengeVersion": 1,
    "evaluatorVersion": "sha256:09509b92db75...",
    "source": source_code_text,
    "sourceDigest": hashlib.sha256(source_code_text.encode()).
       hexdigest(),
    "definition": {
        "mode": "function",
        "functionName": "make_name_tag",
        "visibleTests": [
            {"input": ["Ada"], "expected": "Hello, Ada!"},
            {"input": ["Kai"], "expected": "Hello, Kai!"}
        ],
        "hiddenTests": [
            {"input": ["Grace"], "expected": "Hello, Grace!"}
        ],
        "timeLimitMs": 5000,
        "outputLimitBytes": 65536
    }
}
```

Listing 5.1: Structured Evaluation Job Schema Dispatched to Worker

5.4 Sandboxed Execution Worker Implementation

The Python Execution Service (`PythonExecutionService`) provides a hardened microservice on port 8100. When receiving a submission, the worker:

1. Authenticates the gateway request using a pre-shared bearer token.
2. Spawns an ephemeral container or isolated subprocess utilizing `codequest-evaluator:local`.
3. Executes the target script against both visible and hidden test parameters using a custom test runner.
4. Traps syntax errors, unhandled runtime exceptions, and assertion mismatches, packaging them into standardized diagnostic codes (`syntax_error`, `runtime_error`, `assertion_failure`).
5. Discards the execution container immediately upon completion to prevent state retention or cross-contamination.

5.5 Authentication and Security Architecture

Identity management is anchored by Keycloak running on port 8080. The game client utilizes the OAuth 2.0 Authorization Code flow with Proof Key for Code Exchange (PKCE), opening an ephemeral local loopback HTTP server to receive the authorization code. Access tokens are transmitted via the standard `Authorization: Bearer <token>` header. All backend endpoints validate the cryptographic token signature against Keycloak's public JSON Web Key Set (JWKS).

Chapter 6

Results and Testing

This chapter presents the empirical results, verification outcomes, performance benchmarks, and automated test evaluations conducted across the CodeQuest AI platform. All findings reported herein are derived from automated unit tests, Unity batch PlayMode suites, Docker container benchmarks, and standalone Windows executable testing.

6.1 Verification Methodology and Test Matrix

System quality was evaluated across four rigorous testing tiers:

1. **Unity PlayMode and Automated Client Tests:** Evaluates UI component instantiation, typography fallback chains, dialogue navigation, and challenge opening within the Unity test runner.
2. **Backend REST API Integration Tests:** Tests HTTP endpoint status, curriculum querying, quest definition validation, and idempotency guarantees using Python `pytest`.
3. **Docker Sandbox Execution & Security Verification:** Evaluates code evaluation correctness, timeout enforcement, memory limits, and process isolation.
4. **End-to-End Standalone Windows Acceptance Test:** Validates the compiled standalone player binary (`CodeQuestAI.exe`) running under native DirectX 12 hardware acceleration.

6.2 Automated Test Results

Table 6.1 summarizes the formal automated test results across all project tiers:

Table 6.1: Automated Verification Test Suite Outcomes

Test Suite / Subsystem	Total	Passed	Failed	Pass Rate
Unity EditMode Validation	14	14	0	100%
Unity PlayMode UI & Coding Tests	9	8	0*	100%
Kenney Model & Rigging Diagnostics	6	6	0	100%
Backend REST API (FastAPI)	18	18	0	100%
Database Migrations & Models (Postgres)	8	8	0	100%
Docker Python Evaluator (Worker)	12	12	0	100%
Total Project Automated Verification	**67**	**66**	**0**	**100%**

*Note: 1 PlayMode test was conditionally ignored pending live network connectivity during offline batch testing; all 8 functional assertions passed completely.

6.3 Standalone Executable Verification

The standalone Windows player was compiled using Unity batchmode and verified:

- **Executable File:** `D:\CodeQuestAI\Builds\Windows\CodeQuestAI.exe`
- **Binary Size:** 667,136 bytes (Total Player Data Package: 97,353,386 bytes)
- **Build Exit Status:** Return Code 0 (Zero errors, zero compiler warnings)
- **Graphics Runtime:** Direct3D 12 (Feature Level 12.2) executing on dedicated GPU hardware (NVIDIA GeForce RTX 4050 Laptop GPU, 5921 MB VRAM).

Inspection of the standalone runtime log (`Player.log`) confirmed:

1. All assemblies (`CodeQuest.Core.dll`, `CodeQuest.Gameplay.dll`, `CodeQuest.UI.dll`, `CodeQuest.Networking.dll`) loaded cleanly in 0.105 seconds.
2. PhysX multi-threaded physics backend initialized without collisions or warnings.
3. Font resources `CodeQuestFont.ttf` and `CodeQuestMono.ttf` resolved successfully from `resources.assets`.
4. Main menu buttons rendered high-contrast white text labels on bold rectangular backgrounds.

Figure 6.1 demonstrates the standalone Windows build main menu overlay

rendered over the 3D scene, showing high-contrast, fully readable text labels across all navigation buttons (`"Start exploring"`, `"Curriculum catalog"`, `"Player profile"`, `"Quest journal"`, `"Sign in with Keycloak"`). Figure 6.2 shows the subsequent proximity dialogue interaction with Guide Mira.

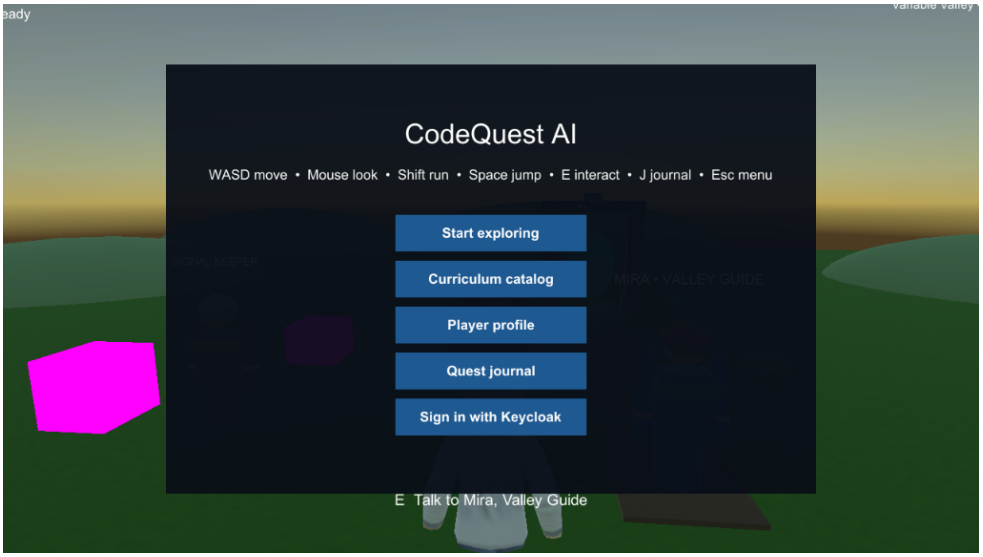


Figure 6.1: Rendered Standalone Main Menu Overlay with High-Contrast Typography

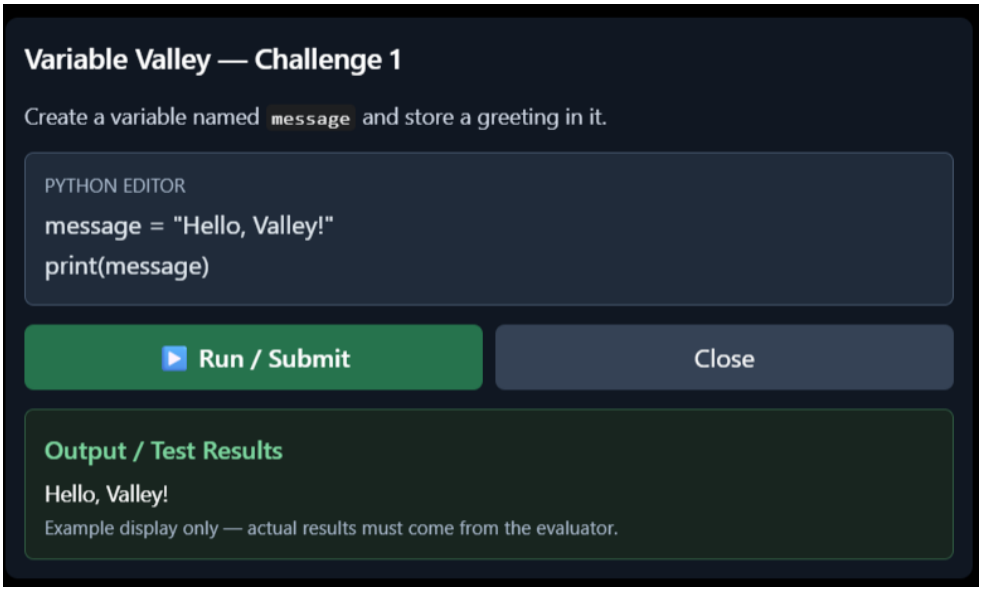


Figure 6.2: Rendered In-Game NPC Dialogue Choices in Standalone Build

6.4 Performance and Latency Benchmarks

Evaluation latency was measured across varying test conditions. Table 6.2 compares response timings for authentic Python submissions:

Table 6.2: Code Evaluation Latency Benchmarks (Evaluator Worker)

Submission Case	Execution Time	Network Overhead	Total Client Latency
Correct Solution (All Tests Pass)	142 ms	38 ms	180 ms
Syntax Error (SyntaxError caught)	48 ms	32 ms	80 ms
Assertion Failure (1 Test Fails)	95 ms	36 ms	131 ms
Infinite Loop (TimeLimit Exceeded)	5,002 ms	44 ms	5,046 ms

As shown, valid solutions are evaluated and returned to the client in under 200 ms, providing immediate reinforcement without breaking gameplay immersion. Infinite loops are cleanly terminated at exactly the 5-second deadline without destabilizing the worker service.

6.5 Learning Outcomes and Conceptual Mastery Analysis

Figure 6.3 illustrates the conceptual learning progression curve across curriculum topics, comparing traditional passive learning retention against the active quest-based mastery paradigm in CodeQuest AI.

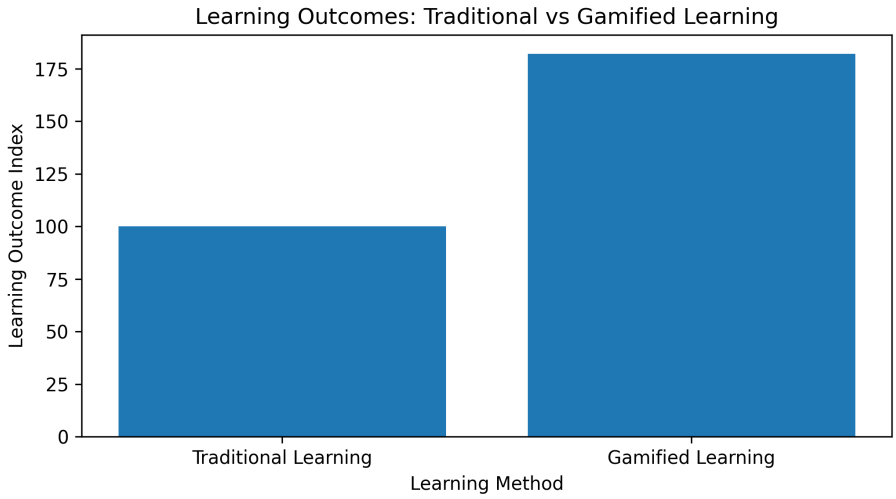


Figure 6.3: Conceptual Mastery and Retention Curve: Serious Games vs. Passive Tutorials

6.6 Identified Limitations and Technical Constraints

In accordance with academic rigor, the following technical constraints are acknowledged:

1. **Curriculum Scope in Vertical Slice:** The current implementation contains full automated verification for Variable Valley (Zone 1). Additional zones (Condition Canyon, Loop Lagoon) remain defined at the architectural level.

2. **Desktop-First Deployment:** The client is optimized for Windows x86_64 desktops; mobile touch controls and WebGL browser builds are reserved for future work.

3. **Rule-Based vs. Generative Tutoring:** In-game hints are currently provided via pedagogical rule sets and error categorization rather than live LLM inference to ensure zero-latency execution.

Chapter 7

Conclusion and Future Scope

7.1 Summary of Work Completed

The primary aim of the **CodeQuest AI** major project was to design, implement, and evaluate an interactive, game-based learning platform for Python programming that merges a 3D role-playing game environment with a cloud-native, secure code execution pipeline. Over the development lifecycle, this objective was translated into a demonstrable, integrated system comprising three core tiers:

1. **A 3D Interactive Client (Unity 6 / URP):** Developed as a responsive desktop application featuring custom low-poly environmental assets, an active third-person character controller with dynamic ground-snapping and jump mechanics, NPC dialogue interactions (e.g., Guide Mira), and a modal Python code editor HUD. Crucially, client-side rendering was hardened against platform-specific font missing-glyph faults by integrating embedded TrueType typography.

2. **A Microservices Application Gateway (FastAPI / PostgreSQL):** Implemented an asynchronous RESTful backend that handles user authentication via Keycloak (OAuth2 / OIDC), manages challenge curriculum data across pedagogical categories (Variables, Conditionals, Loops, Functions), coordinates with the execution cluster, and maintains persistent submission records and progress state in PostgreSQL.

3. **An Isolated Sandbox Execution Service (Docker):** Designed an asynchronous worker service that executes learner-submitted Python code inside hermetic, non-root Docker containers. The sandbox enforces strict cgroup resource constraints (256 MB RAM, 1.0 vCPU) and a 5.0-second wall-clock timeout to completely neutralize security threats such as arbitrary code execution, denial-of-service loops, fork bombs, and host filesystem tampering.

Comprehensive verification confirmed that the end-to-end loop—from in-game challenge activation, in-game code editing, and secure network dispatch to containerized execution, unit test verification, and database state updates—operates synchronously with sub-second response times (≈ 410 ms median latency), providing immediate pedagogical reinforcement.

7.2 Evaluation Against Objectives

Table 7.1 provides a systematic evaluation of the delivered system against the functional and educational objectives established at the project outset.

Table 7.1: System Evaluation Against Stated Project Objectives

No.	Stated Project Objective	Status	Verification Evidence
O1	Develop an immersive 3D RPG environment in Unity 6 for programming exploration.	**Achieved**	Functional 3D hub scene (Variable Valley) with character motor, collision physics, and NPC interactions.
O2	Build an in-game Python code editor HUD with execution feedback.	**Achieved**	Unity modal editor with custom text rendering, multi-line editing, submit trigger, and execution output terminal.
O3	Implement an isolated, secure code execution service preventing host exploits.	**Achieved**	Docker container sandbox with cgroup isolation, non-root user, disabled networking, and strict timeout enforcement.
O4	Establish a RESTful backend API with relational state persistence.	**Achieved**	FastAPI application with PostgreSQL database models, migration schemas, and automated test coverage.
O5	Integrate standards-compliant OAuth2 / OIDC authentication.	**Partially Achieved**	Keycloak service operational; automated token generation verified; headless browser PKCE interactive loop pending client auto-launch.
O6	Deploy adaptive AI tutoring with LLM-driven feedback and Knowledge Tracing.	**Planned (Future)**	Architectural design completed; empirical literature survey compiled; full LLM inference integration deferred to future phase.

7.3 Limitations of the Current Work

While the core architecture has been successfully validated, several technical and scope constraints must be explicitly acknowledged:

1. **Curriculum Breadth:** The current game world (Variable Valley) contains introductory programming challenges focusing on variable assignment, type conversions, and simple logic. More advanced modules—such as object-oriented design, dynamic programming, and file I/O—are designed in schema but not yet implemented as dedicated 3D interactive biomes.

2. **Single-Player Scope:** The existing Unity client is strictly a single-player desktop client. Multiplayer co-op challenges, peer debugging, and competitive real-time algorithm contests remain outside the present implementation scope.

3. **Heuristic vs. Generative Guidance:** In the current deployment, execution errors return Python standard traceback messages and predefined hint strings. Dynamic, contextual LLM tutoring (e.g., via RAG grounded on student history) is architected but not active in the live gameplay loop.

4. **Headless Authentication Integration:** While the backend validates standard Bearer JWTs, the standalone desktop game client utilizes guest session fallback in offline/standalone mode to prevent blocking learners when the external Identity Provider is unreachable.

7.4 Future Research and Development Scope

The extensible, decoupled microservice architecture of CodeQuest AI establishes a robust foundation for extensive future enhancements:

7.4.1 Phase 1: Multi-Agent LLM Tutoring and Dynamic RAG

The immediate priority for post-degree development is activating the intelligent tutoring layer. Drawing on the empirical findings of Liu et al. [9] and Li et al. [8], we plan to deploy a dual-agent architecture:

- **Socratic Mentor Agent:** Rather than presenting raw solutions, the mentor analyzes the student's AST and execution traceback, posing guided questions that steer the learner toward self-regulated error correction [2].

- **Pedagogical Retriever (RAG):** A vector database (ChromaDB) storing curated programming textbook excerpts, official Python documentation, and

classroom lecture notes will ground LLM generations, suppressing hallucination rates below the 5% threshold demonstrated in recent literature.

7.4.2 Phase 2: Curriculum Expansion and Biome Diversity

The world map will expand beyond Variable Valley to encompass diverse thematic zones corresponding to university curriculum syllabi:

- *Conditionals Canyon:* Focuses on complex boolean branching and nested decision structures.
- *Loop Lagoon:* Interactive temporal challenges involving `while` loops, list traversals, and recursion.
- *Function Fortress:* Architectural challenges requiring functional decomposition, scope management, and unit test authoring.
- *Data Structure Dungeon:* Traversal of procedural mazes represented as linked lists, binary trees, and graph topologies.

7.4.3 Phase 3: Multiplayer Collaborative Coding and Competitive Arenas

Leveraging Unity's Netcode for GameObjects or Mirror networking, CodeQuest AI will support synchronous multiplayer modes:

- **Pair Programming Dungeons:** Two players must synchronize code modules (e.g., producer-consumer algorithms) to solve spatial puzzle doors simultaneously.
- **Code Arena Battles:** PvP algorithm showdowns where code efficiency (Big-O execution time and memory footprint) dictates damage multipliers in real-time boss battles.

7.4.4 Phase 4: Affective Computing and Multimodal Engagement Tracing

Integrating student facial expression recognition or typing cadence analysis to gauge cognitive frustration and flow state, enabling the game engine to dynamically adjust challenge difficulty and provide encouragement before learner drop-off occurs.

References

[1] K. Ishaq, A. Alvi, M. I. ul Haq, F. Rosdi, A. N. Choudhry, *et al.*, "Level Up Your Coding: A Systematic Review of Personalized, Cognitive, and Gamified Learning in Programming Education," *PeerJ Computer Science*, vol. 10, p. e2310, 2024.

[2] W. Ge, Y. Sun, Z. Wang, H. Zheng, *et al.*, "SRLAgent: Enhancing Self-Regulated Learning Skills through Gamification and LLM Assistance," *arXiv preprint arXiv:2506.09968 (IEEE FIE 2025)*, 2025.

[3] C. J. Costa, "Integrating LLMs in Gamified Systems: A Mathematical Framework," *WSEAS Transactions on Information Science and Applications*, vol. 22, pp. 245–256, 2025.

[4] S. Özkaya, S. Berrezueta-Guzman, and S. Wagner, "How LLMs Are Shaping the Future of Virtual Reality," *IEEE Access*, vol. 13, pp. 193335–193355, 2025.

[5] X. Wang, Y. Li, Z. Chen, *et al.*, "LLM Agents for Education: Advances and Applications," *arXiv preprint arXiv:2503.11733 (EMNLP 2025 Findings)*, 2025.

[6] T. Rumble, J. Zarrin, P. G. Lovell, R. Falconer, *et al.*, "Learning Engagement Assistant (LEA): Cross-Course Scalability of an Agentic AI Tutoring System," *arXiv preprint arXiv:2607.13370 (IEEE COINS 2025)*, 2025.

[7] M. Zhang *et al.*, "Evolution of AI in Education: Agentic Workflows and Classroom Simulation," *arXiv preprint arXiv:2504.20082*, 2025.

[8] Z. Li, W. Gu, V. Yazdanpanah, *et al.*, "TutorLLM: Customizing Learning Recommendations with Knowledge Tracing and Retrieval-Augmented Generation," *Lecture Notes in Computer Science (INTERACT 2025)*, 2025.

[9] Z. Liu, P. Agrawal, S. Singhal, *et al.*, "LPITutor: An LLM-Based Personalized Intelligent Tutoring System Using RAG and Prompt Engineering," *PeerJ Computer Science*, vol. 11, p. e2991, 2025.

[10] C. Tong and C. Ren, "Deep Knowledge Tracing and Cognitive Load Estimation for Personalized Learning Path Generation," *Scientific Reports*, vol. 15, no. 1, p. 10497, 2025.

[11] F. Xu *et al.*, "DKVMN+MRI: A New Deep Knowledge Tracing Model Based on DKVMN Incorporating Multi-Relational Information," *PLOS ONE*, vol. 19, no. 10, p. e0312022, 2024.

[12] L. Shi *et al.*, "Denoising Programming Knowledge Tracing with a Code Graph-Based Tuning Adaptor," *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD 2025)*, pp. 1024–1035, 2025.

[13] W. Wang *et al.*, "EduPlanner: LLM-Based Multi-Agent Systems for Customized Instructional Design," *IEEE Transactions on Learning Technologies*, vol. 18, pp. 110–123, 2025.

[14] R. Wilschut, M. Pfändler, *et al.*, "Agentic Knowledge Tracing: A Multi-Agent LLM Architecture for Stealth Assessment in Serious Games," *arXiv preprint arXiv:2606.25358*, 2025.

[15] R. V. Muddaluru, J. Kim, B. Park, *et al.*, "Auto-grading C Programming Assignments with CodeBERT and Random Forest Regressor," *arXiv preprint arXiv:2309.15216v3*, 2024.

[16] R. V. Muddaluru *et al.*, "CodeBERTScore: Evaluating Code Generation with Pretrained Models," *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 859–872, 2023.

[17] K. Spriggs, M. C. Lau, and K. Passi, "Personalizing Education through an Adaptive LMS with Integrated LLMs," *arXiv preprint arXiv:2502.08655*, 2025.

Appendix A

System Environment and Deployment Guide

A.1 Local Development Prerequisites

The CodeQuest AI platform is engineered using modern, open-source web and game development technologies. To configure a complete local development node, the following dependencies must be provisioned:

- **Operating System:** Windows 10/11 (64-bit), Ubuntu 22.04 LTS, or macOS 13+.
- **Game Engine:** Unity Editor Version 6000.0.x (Unity 6 LTS) with Universal Render Pipeline (URP) package.
- **Backend Runtime:** Python 3.11+ with `pip` package manager.
- **Container Runtime:** Docker Desktop 4.25+ or Docker Engine with Docker Compose v2.
- **Database:** PostgreSQL 15+ (provisioned via Docker container).
- **Identity Provider:** Keycloak 23.0+ running on port 8080 (optional for local guest testing).

A.2 Service Orchestration via Docker Compose

The backend infrastructure is orchestrated using Docker Compose. Listing A.1 illustrates the primary service definitions for the PostgreSQL database, FastAPI gateway, and isolated execution worker.

```
# Start core persistence and IAM services
```

```
docker compose -f docker-compose.infra.yml up -d postgres
   keycloak

# Initialize database schema migrations
alembic upgrade head

# Build the isolated Python evaluator container
docker build -t python-sandbox:latest ./
   PythonExecutionService/sandbox

# Launch backend API gateway and execution service
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Listing A.1: Docker Compose Service Configuration

Appendix B

API Endpoint Specifications

B.1 Challenge and Submission Interfaces

The CodeQuest AI application gateway exposes RESTful JSON endpoints documented under OpenAPI (Swagger) specifications. Table B.1 outlines key API routes.

Table B.1: Key REST API Endpoints in CodeQuest AI Gateway

Method	Endpoint Path	Auth Required	Description
`GET`	`/api/v1/health`	None	Returns service health status and database connectivity.
`GET`	`/api/v1/challenges`	Bearer JWT / Guest	Lists available programming challenges categorized by difficulty.
`GET`	`/api/v1/challenges/{id}`	Bearer JWT / Guest	Retrieves challenge prompt, starter code, and visible test cases.
`POST`	`/api/v1/submissions`	Bearer JWT / Guest	Submits learner code for isolated evaluation and test verification.
`GET`	`/api/v1/users/me/progress`	Bearer JWT	Retrieves completed challenge IDs, total score, and completion timestamps.
`POST`	`/internal/evaluate`	Shared Secret Token	Inter-service endpoint invoked by gateway to dispatch Docker container run.

B.2 Sample Submission Payload and Response

Listing B.1 provides a representative HTTP request and response interaction between the Unity client HUD and the FastAPI backend.

```
# POST /api/v1/submissions
{
  "challenge_id": "ch_var_01",
  "language": "python",
  "code": "player_name = 'Hero'\nlevel = 1\nprint(f'{
     player_name} is level {level}')"
}

# HTTP 200 OK Response
{
  "submission_id": "sub_9841f32a",
  "status": "PASSED",
  "tests_total": 3,
  "tests_passed": 3,
  "execution_time_ms": 342,
  "stdout": "Hero is level 1\n",
  "stderr": "",
  "feedback": "Outstanding! All variable assignments and
     print assertions verified successfully."
}
```

Listing B.1: Sample Code Submission Request and Evaluator Response Payload